

VedicOS – PCB Hardware Requirements

Bare Minimum Version + Optional Extensions

1. Basic Hardware

The device is a very small, pocket-sized offline Vedic companion.

Main components

- **Display:** 1.54" 200×200 monochrome E-Ink display
- **MCU:** Seeed XIAO ESP32-S3 / ESP32-S3 module
- **Battery:** 400 mAh Li-Po as the current target
- **Vibration motor:** Required
- **Direction sensor:** Small magnetometer/compass sensor – required
- **Buttons:** 3 physical side buttons
- **Power switch:** 1 small ON/OFF slide switch
- USB charging for Li-Po battery
- PCB should be designed to be as compact as practically possible.

Physical size

The device should **not be larger than the 1.54" E-Ink display footprint by much**.

The target is approximately the same overall length/height as the 1.54" display, with the PCB and components arranged around/behind it.

The three buttons and the small power/Japa safety switch should be mounted on the **side of the device**.

The final PCB should be designed with the smallest practical footprint.

2. BASIC VERSION – Required

This is the minimum version that must work.

Controls

There will be:

Button 1 – Mode button

- Short/normal press: switch between screens/modes.
- Long press: switch between the main operating modes.

Button 2 – Dedicated Japa button

- Used specifically for Japa counting.
- I will handle the counting logic in ESP32 firmware.
- The ESP32 will count button presses.
- The E-Ink display will only refresh after a predefined number of counts to save battery.
- Vibration feedback will occur at every 108 counts.
- This button should be physically easy to operate with the thumb.

Button 3 – Auxiliary button

- Reserved for additional functions.
- In the optional audio version, long-pressing this button will start the audio/sleep mode.

Small side switch

A separate small **ON/OFF switch is required for Japa mode safety.**

Japa counting will only work when this switch is ON.

The purpose is to prevent accidental Japa counting when the device is inside a pocket or being carried.

3. Software Modes

I will write the ESP32 firmware myself.

For the initial PCB testing, the firmware can simply display:

- Screen 1 → "Hello World 1"
- Screen 2 → "Hello World 2"
- Screen 3 → "Hello World 3"

The three screens should be switchable using the buttons.

Main modes

Mode 1 – Panchang

I will develop the Panchang software myself.

It will eventually display things such as:

- Tithi
- Nakshatra
- Masa

- Hora
- Sunrise/sunset
- Other Panchang information

Mode 2 – Photo

I will write the firmware for this.

The ESP32 will temporarily enable Wi-Fi, allow a photo to be sent to the device, convert/display it on the E-Ink screen, and remain on the photo screen until the user changes the mode.

Mode 3 – Japa

Dedicated Japa operation.

The ESP32 counts the Japa button presses and updates the display only when required.

The display should not refresh after every press because E-Ink refreshes consume power and are unnecessary.

Vibration should occur at every 108 counts.

4. Required Sensors / Hardware

The following are mandatory for the basic PCB:

Vibration

A small vibration motor with suitable transistor/MOSFET driver circuitry.

Direction / Compass

A small onboard magnetometer/compass sensor.

The exact sensor can be selected based on:

- Size
- Power consumption
- Availability
- Price
- ESP32 compatibility

I will handle the software side.

5. OPTIONAL VERSION – Audio

If it is physically possible without making the device significantly larger, I would like to integrate a small speaker.

Audio hardware

- Small onboard speaker
- Required amplifier/driver circuitry
- Small **16 MB SPI flash memory chip**

The flash will store a small audio/song file.

The intended operation:

Long press Button 3 → start audio mode

The device plays the stored song.

When the song finishes, the firmware automatically returns to the home screen.

I will write the audio playback/software logic.

This version is optional. The priority is keeping the device extremely small.

6. OPTIONAL / LUXURY VERSION – GPS

If a sufficiently small and low-power GPS/GNSS module can fit, I would like to consider adding it.

The purpose is not continuous GPS tracking.

The ESP32 would wake the GPS periodically, for example every X minutes, obtain the location, and then turn the GPS off again.

I already have software that can:

1. Read GPS coordinates.
2. Compare them against temple/location data stored in flash.
3. Determine whether a temple is nearby.

The temple database would be stored locally in flash.

The GPS should therefore be designed with **very low power consumption**, since it will normally remain OFF.

7. Power / Battery

Current target:

400 mAh Li-Po rechargeable battery

The device will spend most of its time in a low-power/sleep state.

The E-Ink display retains its image without continuous power, and most modes will not require continuous operation.

Therefore battery life is a major priority.

If a slightly larger battery can physically fit without making the device significantly larger, I am open to that.

Important power requirements

Please design for:

- Very low ESP32 sleep current
- Ability to completely power down unnecessary peripherals
- GPS completely OFF when not being used
- Speaker/audio circuitry OFF when not being used
- Vibration motor OFF except during vibration events
- Efficient Li-Po charging
- Proper battery protection
- E-Ink powered only when required

RTC / clock backup

I was considering a separate coin-cell battery for the RTC and possibly GPS-related operation.

Please advise whether this is actually necessary.

If an RTC with coin-cell backup can be integrated economically and within the size constraint, I am open to it.

For GPS, I would prefer not to use a separate battery unless there is a clear technical reason.

8. PCB Design Priority

Please prioritize the requirements in this order:

1. **Very small physical size**
2. 1.54" E-Ink integration
3. ESP32-S3
4. 3 side buttons
5. Japa safety ON/OFF switch
6. Vibration motor
7. Compass/magnetometer
8. Li-Po charging and battery management
9. Low-power design
10. Optional speaker + 16 MB flash
11. Optional GPS

The **basic version should be designed first.**

The speaker/flash and GPS should be treated as optional extensions only if they can fit without compromising the size, battery life, or reliability of the basic device.

9. What I Need From the PCB Designer

For the first stage, please provide:

1. **Block diagram**
2. Proposed component selection
3. PCB size/layout proposal
4. ESP32-S3 + E-Ink + buttons + switch + vibration + compass circuit
5. Battery charging/protection circuit
6. Power architecture
7. Approximate current consumption
8. Suggestions for making the PCB smaller
9. Whether the optional speaker/flash can fit
10. Whether a small GPS module can realistically fit

I will handle the **ESP32 firmware, Panchang calculations, display UI, Japa logic, photo/Wi-Fi logic, GPS temple detection and audio software.**

The main requirement from the PCB side is to make the hardware **as small, power-efficient and reliable as possible.**